

Aarhus University

Project number - 2026F25

Progression System

Students:

Thea Teresa Lindgaard (202204350)

Niels Jakob (202105879)

Kasper Stecher (202100451)

Kasper Bentsen (202103249)

Counsellor:

Lukas Esterle

Aarhus, May 29th 2026

Contents

1. Introduction	1
1.1. Requirements	1
2. Design	2
2.1. Architecture	2
2.1.1. C4	3
2.1.2. Design Patterns	3
2.2. Classes	3
2.3. Synchronization Strategy	4
2.4. Handling Assymetric Gameplay	5
2.5. Testing Strategy	5
3. Implementation	6
4. ProgressManager Functions	7
4.1. Save()	7
4.2. Load()	7
4.3. Encrypt(text)	7
4.4. Decrypt(encrypted)	8
4.5. GetLevel(levelName)	8
4.6. RegisterLevelCompletion(levelName, role, completionTime, devTime)	8
5. Testing Execution	9
5.1. Functional Testing	9
5.2. Integration Testing	9
5.3. Manual Playtesting	9
5.4. Observations	9
5.5. Summary	10
6. Conclusion	11
Bibliography	12

1. Introduction

Author: Kasper Bentsen

The Progression System is responsible for tracking player achievements and performance across game sessions. It provides persistent storage of level-specific data, including completion times and trophy unlocks, allowing players to track their progress and replay levels for improved results.

The system is designed to operate locally on each client and integrates with gameplay systems to automatically record progress upon level completion.

1.1. Requirements

The requirements, as outlined in **Appendix 1.c.**, are:

Functional

- 1) Level Timer (**E3-US1**)
- 2) Time Trial (**E3-US2**)
- 3) Level Progression (**E6-US1**)
- 4) Auto Save (**E6-US2**)
- 5) Load Game Progress (**E6-US3**)
- 6)

Non-functional

- None

These requirements serve as a guideline to ensure re-playability, game challenge and player retention. The progression system is not related to any of the non-functional requirements.

2. Design

The following section provides a detailed description of the design of the progression system.

2.1. Architecture

The Progression System follows a centralized architecture based around a static manager responsible for data access, persistence, and updates. All progression-related operations are handled through this manager, ensuring a single point of control.

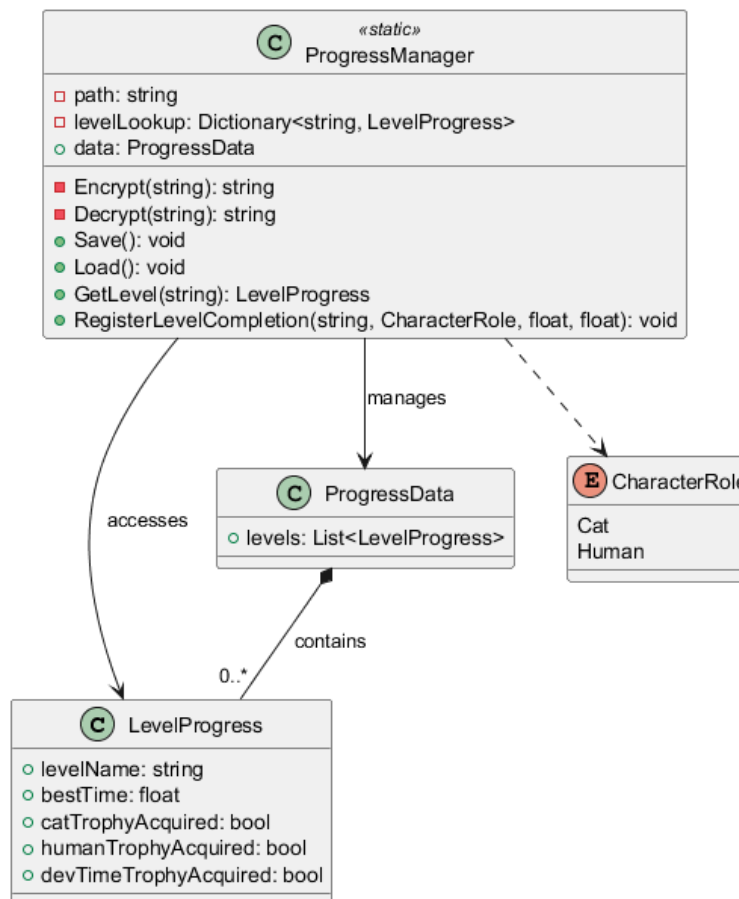


Figure 1: Class diagram explaining the relationship between the classes used to store progress data

Progress data is stored as a collection of level-specific entries, each representing the player's performance and achievements for a given level. This structure is displayed in the class diagram in **Figure 1**.

2.1.1. C4

At a high level, the system operates entirely on the client side and interacts with the following components:

- LevelManager (provides completion data)
- File system (persistent storage)

The relationship between the components is demonstrated in **Figure 2**.

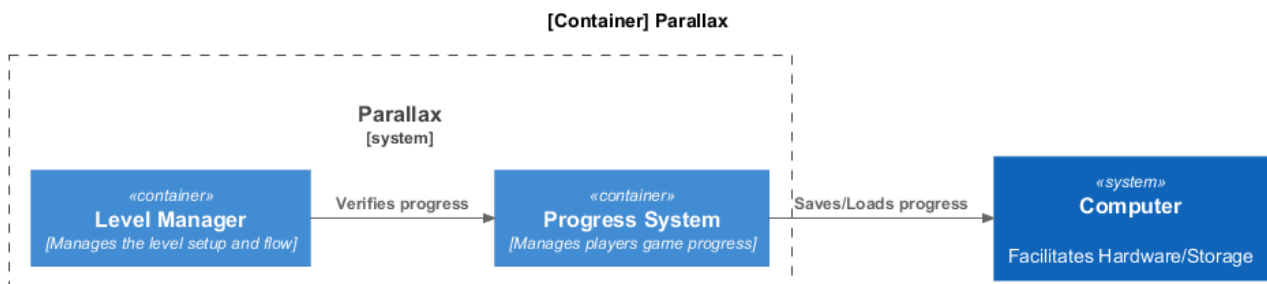


Figure 2: Class diagram explaining the relationship between the classes used to store progress data

The system does not depend on external services or networking and is therefore isolated from the multiplayer infrastructure.

2.1.2. Design Patterns

The system utilizes the following design patterns:

- **Singleton Pattern:** The ProgressManager acts as a globally accessible static instance managing all progression data.
- **Data Container Pattern:** ProgressData and LevelProgress are simple serializable classes used to structure stored data.
- **Lazy Initialization:** Level progress entries are created on demand when accessed for the first time.

2.2. Classes

The core classes of the system include:

- **ProgressManager** Handles saving, loading, encryption, and updating of progression data.
- **ProgressData** Represents the root data structure containing a list of all level progress entries.
- **LevelProgress** Stores per-level data such as best completion time and trophy states.

Other classes used by or using the system:

- **LevelData** Defines static level metadata such as name, developer time, and associated scene.
- **LevelTile** Responsible for presenting progression data in the UI.
- **LevelManager** Provides runtime data such as completion time and player role upon level completion.

2.3. Synchronization Strategy

The Progression System operates entirely locally and does not synchronize data between clients. Each game client maintains their own progression data independently.

During gameplay, progression is only updated when a level is completed. The LevelManager triggers this update through a client RPC, ensuring that each client registers their own completion data based on their local state.

This approach avoids conflicts and simplifies implementation, as no shared progression state needs to be maintained across the network.

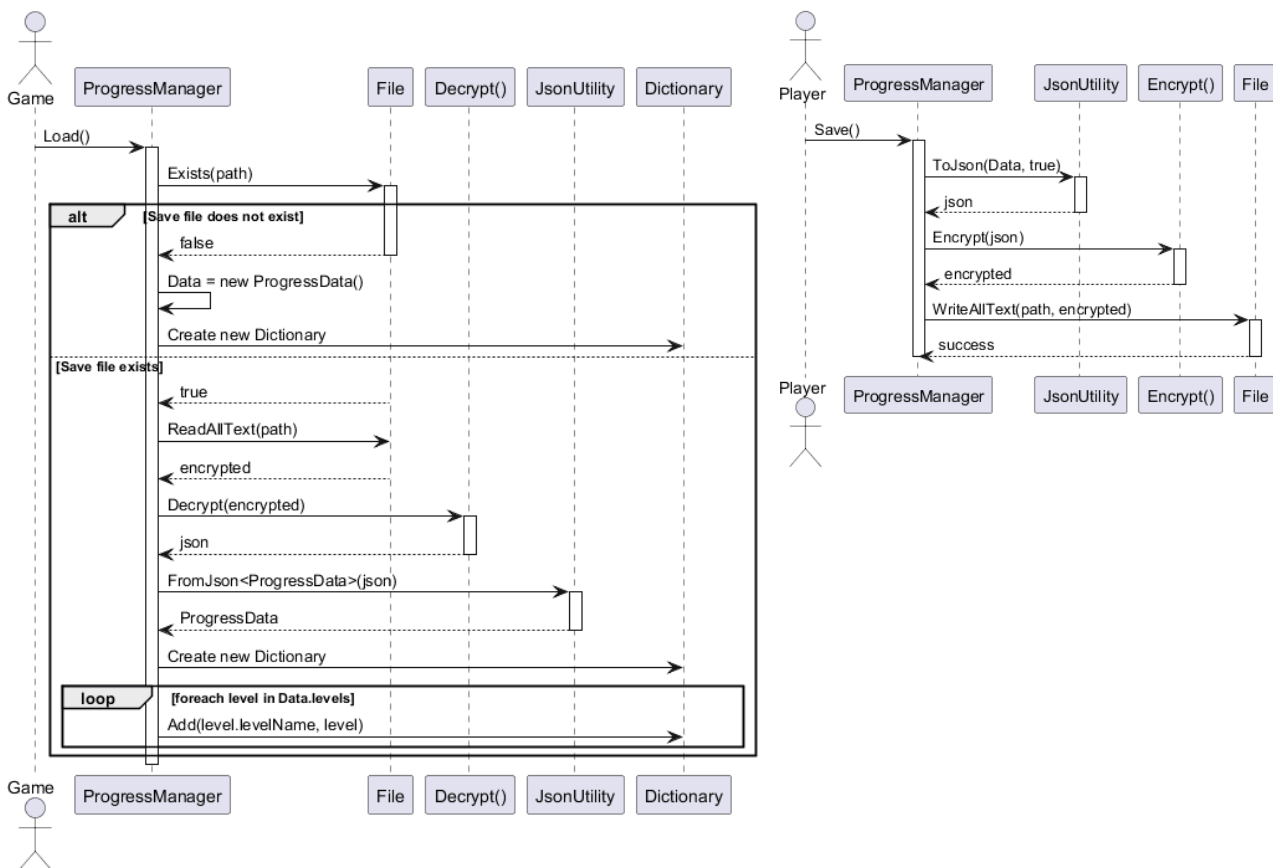


Figure 3: Sequence diagram demonstrating the process of loading(left) and saving(right) data via the progression system to a file store locally on the computer

The diagrams in **Figure 3** explain the sequence of loading and saving data via the ProgressManager.

While loading the data, if a save file does not already exist, the ProgressData is initialized in the ProgressManager and updated with LevelProgress accordingly. If the file exists, its' data is decoded and loaded to the ProgressManager, where each level is added to the levelLookup dictionary, for cached access.

2.4. Handling Assymmetric Gameplay

The system explicitly supports asymmetric gameplay by tracking progression separately for each player role.

Each level stores:

- A trophy for completing the level as the cat
- A trophy for completing the level as the human

This ensures that both roles are encouraged to be played and completed independently.

Additionally, a developer-defined time trial, awards a trophy as well, introducing an additional challenge that aims to boost player engagement and re-playability.

2.5. Testing Strategy

The Progression System was tested through a combination of functional testing, integration testing, and manual playtesting.

Functional testing focused on verifying correct data storage and retrieval. Integration testing validated interactions between gameplay systems and the progression manager. Manual playtesting ensured that progression behaved correctly during real gameplay scenarios.

3. Implementation

The Progression System is implemented as a static manager responsible for handling all progression-related operations.

Progress data is serialized using Unity's built-in JSON utility[1] and stored locally on disk. Before storage, the data is obfuscated using a simple XOR-based encryption[2], [3], [4] and encoded as Base64. This prevents direct readability while maintaining low computational overhead. Since the stored data is for personal progression tracking only, it was unnecessary with actual encryption and the obfuscation tactic was deemed sufficient.

Upon loading, the system reconstructs the progression state and builds a lookup dictionary for efficient access to level-specific data.

When a level is completed, the system performs the following steps:

- Retrieves or creates the corresponding level entry
- Updates the best completion time if improved
- Registers role-specific completion trophies
- Checks and assigns a developer-time trophy
- Saves the updated data to disk if changes occurred

UI elements dynamically read progression data to display:

- Best completion time
- Developer time
- Trophy states

This ensures that the UI always reflects the latest progression state, as demonstrated in **Figure 4**, without requiring manual updates.



Figure 4: The “level tile” used in game during level selection, displays all three trophies in the lower right corner of the level image. Below the image, the users best time can be seen to the left, and the developer set trial time can be seen to the right.

4. ProgressManager Functions

4.1. Save()

Purpose: Saves the current progress data to persistent storage.

Description:

- Serializes the current ProgressData object into JSON format.
- Encrypts the serialized data.
- Writes the encrypted data to a file located in Unity's persistent data path.
- Ensures player progress is preserved between game sessions.

4.2. Load()

Purpose: Loads saved progress data from persistent storage.

Description:

- Checks whether a save file exists.
- If no file exists:
 - Creates a new empty ProgressData instance.
 - Initializes a new lookup dictionary.
- If a save file exists:
 - Reads the encrypted file contents.
 - Decrypts the data.
 - Deserializes the JSON into a ProgressData object.
 - Rebuilds the level lookup dictionary for fast access.

4.3. Encrypt(text)

Purpose: Encrypts save data before writing it to disk.

Description:

- Converts the input text into UTF-8 bytes.
- Applies XOR encryption using a fixed byte key.
- Encodes the encrypted bytes as Base64.
- Returns the encrypted string representation.

4.4. Decrypt(encrypted)

Purpose: Decrypts previously encrypted save data.

Description:

- Decodes the Base64 string into bytes.
- Applies XOR decryption using the same byte key.
- Converts the decrypted bytes back into a UTF-8 string.
- Returns the decrypted JSON data.

4.5. GetLevel(levelName)

Purpose: Retrieves progress data for a specific level.

Description:

- Searches the lookup dictionary for the requested level.
- If the level exists:
 - Returns the existing `LevelProgress` object.
- If the level does not exist:
 - Creates a new `LevelProgress` instance.
 - Assigns the provided level name.
 - Adds the level to both the progress list and lookup dictionary.
 - Returns the newly created object.

4.6. RegisterLevelCompletion(levelName, role, completionTime, devTime)

Purpose: Registers completion results for a level.

Description:

- Retrieves the level progress entry.
- Compares the completion time against the saved best time.
- Updates the best time if a faster completion is achieved.
- Unlocks role-specific trophies depending on the character used.
- Unlocks the developer time trophy if the completion time meets the target requirement.
- Saves the updated progress data if any changes were made.

5. Testing Execution

The Progression System was tested iteratively throughout development to ensure correct behavior across both gameplay and persistence scenarios.

5.1. Functional Testing

Functional testing focused on verifying correct data handling and persistence.

The following scenarios were tested:

- Saving progression data after level completion
- Loading progression data on application start
- Correct updating of best completion time
- Correct assignment of role-specific trophies
- Correct assignment of developer-time trophies
- Creation of new level entries when none exist

Edge cases such as first-time level completion and repeated completions were also tested.

5.2. Integration Testing

Integration testing validated interaction between the Progression System and gameplay systems.

The following interactions were tested:

- Triggering progression updates from the LevelManager
- Correct role detection and registration
- Accurate timing using the level stopwatch
- UI updates reflecting newly acquired progression data

Special attention was given to ensuring that progression updates occurred only after level completion and that no duplicate or incorrect entries were created.

5.3. Manual Playtesting

Manual playtesting was conducted to evaluate the system under realistic gameplay conditions.

Test scenarios included:

- Completing levels with different roles
- Replaying levels to improve completion times
- Achieving developer-time thresholds
- Restarting levels and verifying that progress was not prematurely saved
- Exiting and relaunching the game to verify persistence

5.4. Observations

Testing revealed that the system behaved consistently across all tested scenarios.

The use of a lookup dictionary significantly improved access performance for level data. Additionally, conditional saving prevented unnecessary file operations, reducing overhead.

The encryption mechanism provided basic obfuscation but is not intended as a secure solution against malicious actors.

No critical issues were observed during normal gameplay or persistence operations.

5.5. Summary

The Progression System was successfully validated through a combination of functional, integration, and manual testing. The system reliably stores, updates, and retrieves progression data while integrating seamlessly with gameplay systems.

6. Conclusion

The Progression System provides a functional and reliable solution for tracking player performance and trophies across game sessions. It supports the core requirements of the project by enabling persistent storage of level completion data and role-specific progression.

The system integrates effectively with existing gameplay systems and operates independently of the multiplayer infrastructure, ensuring simplicity and robustness.

While the current implementation prioritizes simplicity and efficiency, it provides a solid foundation for potential future extensions, such as cloud-based synchronization or more advanced progression mechanics.

Bibliography

- [1] Unity, "JSON Serialization." [Online]. Available: <https://docs.unity3d.com/2022.3/Documentation/Manual/JSONSerialization.html>
- [2] Multiple, "What is XOR Encryption." [Online]. Available: <https://stackoverflow.com/questions/2029426/what-is-xor-encryption>
- [3] unknown, "XOR Encryption Algorithm." [Online]. Available: <https://www.101computing.net/xor-encryption-algorithm/>
- [4] Multiple, "XOR Cipher." [Online]. Available: https://en.wikipedia.org/wiki/XOR_cipher